

TRATAMENTO DE ERROS EM JAVASCRIPT

`try` • `catch` • `throw` • `Error` • `finally`

🔗 1. O que é tratamento de erros?

Em um programa ou teste automatizado, podem ocorrer situações inesperadas, como um elemento que não existe, uma operação inválida ou uma falha durante a execução.

O **tratamento de erros** permite identificar essas situações, capturar o erro e executar ações apropriadas, evitando que o programa simplesmente pare sem fornecer informações úteis.

No contexto de **QA e testes automatizados**, isso ajuda a tornar os testes mais confiáveis e facilita a identificação das falhas.

◆ 2. `try` — TENTAR

O bloco `try` contém o código que **pode provocar um erro**.

```
try {  
    // código que pode gerar erro  
}
```

Para lembrar:

🔗 `try` = **tente executar**.

◆ 3. `catch` — CAPTURAR

O `catch` é executado quando ocorre um erro dentro do `try`. Ele permite **capturar e tratar o problema**.

```
try {  
    executarTeste();  
} catch (erro) {  
    console.log(erro.message);  
}
```

A variável `erro` contém informações sobre o erro.
`erro.message` apresenta a **mensagem do erro**.

Para lembrar:

🔗 `catch` = **captura o erro**.

◆ 4. `throw` — GERAR UM ERRO

O comando `throw` permite **gerar um erro manualmente** quando uma condição inválida é encontrada.

```
if (!elemento) {  
    throw new Error("Elemento não encontrado!");  
}
```

Isso é muito útil em testes, pois podemos determinar explicitamente que determinada situação deve ser considerada uma falha.

Para lembrar:

🔗 `throw` = **lança/gera um erro**.

◆ 5. `Error` — REPRESENTAR O ERRO

`Error` é utilizado para criar um objeto que representa um erro.

```
throw new Error("Usuário não encontrado!");
```

A mensagem pode ser acessada através de:

```
erro.message
```

Exemplo:

```
Usuário não encontrado!
```

◆ 6. `finally` — FINALIZAR

O bloco `finally` é executado **independentemente de ter ocorrido um erro ou não**.

```
try {
  executarTeste();
} catch (erro) {
  console.log(erro.message);
} finally {
  limparDados();
}
```

É muito utilizado para ações de **limpeza (cleanup)**, como:

- Fechar navegador;
- Limpar dados de teste;
- Encerrar recursos;
- Finalizar uma operação.

Para lembrar:

☞ `finally` = **sempre finalizar/limpar**.

☐ 7. ESTRUTURA COMPLETA

```
try {
  // tenta executar
}
catch (erro) {
  // trata o erro
}
finally {
  // finaliza/limpa
}
```

🔥 MACETE PARA A PROVA

TRY TENTA → CATCH CAPTURA → FINALLY FINALIZA

E:

THROW LANÇA um ERROR.

🔧 8. EXEMPLO PRÁTICO EM QA

```
try {
  validarLogin();
  validarPaginaInicial();
} catch (erro) {
```

```
        console.log("Teste falhou:", erro.message);

    } finally {

        limparDadosDeTeste();
        fecharNavegador();
    }
}
```

O que acontece?

- 1. `try` → tenta executar as validações.
- 2. **Se ocorrer erro** → o `catch` captura o problema.
- 3. `erro.message` → mostra o motivo da falha.
- 4. `finally` → executa a limpeza mesmo após uma falha.

★ RESUMO FINAL

Conceito	O que faz?
<code>try</code>	Tenta executar um código
<code>catch</code>	Captura/trata um erro
<code>throw</code>	Gera ou lança um erro
<code>Error</code>	Cria um objeto de erro
<code>erro.message</code>	Obtém a mensagem do erro
<code>finally</code>	Executa a finalização/limpeza

🎓 PARA MEMORIZAR

- `try` → **tentar**
- `catch` → **capturar**
- `throw` → **lançar**
- `Error` → **erro**
- `finally` → **finalizar**

O objetivo principal do tratamento de erros é **controlar falhas, obter informações úteis sobre elas e garantir que ações importantes de finalização sejam realizadas.**